stage2_claude_code_deep_analysis

Deep Analysis: Claude Code (claude-code-source)

Executive Summary

Claude Code is a CLOSED-SOURCE AI CLI agent distributed by Anthropic as a native binary. The GitHub repository (`anthropics/claude-code`) is a **documentation and plugin repository**, NOT the source code. The actual application is a JavaScript/TypeScript application compiled to native binaries using **Bun's `--compile` feature**, distributed via npm as platform-specific packages (`@anthropic-ai/claude-code-darwin-arm64`, etc.).

Despite being closed-source, this analysis extracts comprehensive architectural intelligence from: - **SDK Type Definitions** (`sdk-tools.d.ts`) - Complete tool API schemas - **Binary String Analysis** - Internal architecture, bundled dependencies, runtime - **CHANGELOG** (3412 lines, 120+ releases) - Complete feature evolution - **Plugin Examples** - Plugin architecture, hooks system, skills format - **Settings Examples** - Configuration system, sandboxing, permissions - **Installation Scripts** - Distribution mechanism, platform detection

---

1. Repository Structure

What the GitHub Repository Actually Contains

```
claude-code/
├── .claude/                    # Example project-level commands
│   └── commands/               # Custom slash command definitions
│       ├── commit-push-pr.md
│       ├── dedupe.md
│       └── triage-issue.md
├── .claude-plugin/             # Plugin metadata for this repo
│   └── marketplace.json
├── .devcontainer/              # Dev container configs
├── .github/                    # GitHub workflows (issue triage, etc.)
│   ├── ISSUE_TEMPLATE/
│   └── workflows/              # 15+ automated workflows
├── .vscode/                    # VS Code extension settings
├── examples/                   # Configuration examples
│   ├── hooks/                  # Hook validator examples
│   ├── mdm/                    # Mobile device management configs
│   └── settings/               # settings.json examples
│       ├── settings-strict.json
│       ├── settings-lax.json
│       └── settings-bash-sandbox.json
├── plugins/                    # OFFICIAL PLUGIN EXAMPLES (12 plugins)
│   ├── agent-sdk-dev/
│   ├── claude-opus-4-5-migration/
```

```
│   ├── code-review/
│   ├── commit-commands/
│   ├── explanatory-output-style/
│   ├── feature-dev/
│   ├── frontend-design/
│   ├── hookify/
│   ├── learning-output-style/
│   ├── plugin-dev/
│   ├── pr-review-toolkit/
│   ├── ralph-wiggum/
│   └── security-guidance/
├── scripts/                    # Automation scripts (TypeScript)
└── CHANGELOG.md               # 3412 lines of release notes
```

The Actual Binary Distribution

```
@anthropic-ai/claude-code (npm wrapper)
├── bin/claude.exe              # Placeholder stub
├── cli-wrapper.cjs             # Fallback Node.js launcher
├── install.cjs                 # Postinstall binary placement
├── sdk-tools.d.ts              # Complete tool API schemas
└── node_modules/
    └── @anthropic-ai/claude-code-linux-x64/
        └── claude              # 248MB native binary
```

Platform Support: macOS (ARM64, x64), Linux (x64, ARM64, musl), Windows (x64, ARM64)

---

2. Core Architecture

Runtime: Bun-Compiled Native Binary

The 248MB `claude` binary is a **Bun-compiled single executable** containing: - Complete V8/Node.js JavaScript runtime (evidenced by `napi_*`, `uv_*`, `v8::` symbols) - Bundled application code (obfuscated/minified JavaScript visible in strings) - Embedded static assets, help text, completion scripts - Native module dependencies (SQLite, crypto, networking)

Key Evidence from Binary Strings: - `bun getcompletes`, `Bun.serve`, `Bun.stripANSI` - Bun runtime APIs - `bun_install_boolean_flags`, `bun-grouped` - Bun's internal shell completion - `--hot`, `--watch`, `--preload` - Bun CLI flags embedded - `internal:streams/destroy`, `builtin://node/stream/duplex` - Node.js compatibility layer

Internal Message Architecture

From binary string analysis, Claude Code uses a normalized message format:

```
// Message types extracted from binary
{type: "assistant", message: {content: [...],
        context_management: {...}}, uuid, timestamp, ...}
{type: "user", message: {content: [...]}, toolUseResult,
        mcpMeta, ...}
```

```
{type: "progress", data, toolUseID, parentToolUseID, uuid,
        timestamp}
{type: "attachment", attachment: {type: "hook_*", hookEvent,
        hookName, toolUseID}}
{type: "system", subtype: "api_error|local_command"}
```

Context Management Field: `context_management` attached to assistant messages - this is the key to how Claude Code handles large contexts.

Multi-Model Architecture

Claude Code supports multiple provider backends: - **Anthropic API** (primary) - **Amazon Bedrock** (with Mantle support, SigV4 auth) - **Google Vertex AI** (with interactive setup wizard) - **Azure/OpenAI-compatible gateways** (via `ANTHROPIC_BASE_URL`) - **Claude.ai Console** (OAuth-based)

Model Selection: `sonnet` | `opus` | `haiku` for subagents, with effort levels (`low`, `medium`, `high`, `max`)

---

3. Tool Use Framework

Complete Tool Inventory (from sdk-tools.d.ts)

Claude Code exposes **18 core tools** to the LLM:

File Operations

Tool	Description	Key Features
Read (`FileRead`)	Read files with pagination	`offset`, `limit`, PDF page ranges, image display, notebook cells
Write (`FileWrite`)	Create/overwrite files	Absolute paths, git diff output, structured patch
Edit (`FileEdit`)	Search-and-replace edits	`old_string`/`new_string`, `replace_all`, user modification tracking
NotebookEdit	Jupyter notebook editing	Cell CRUD, code/markdown types

Search & Discovery

Tool	Description	Key Features
Grep (`GrepInput`)	ripgrep-based search	`output_mode`: content/files/count, `-A`/`-B`/`-C`, `head_limit`, `offset`, `multiline`
Glob (`GlobInput`)	File globbing	Pattern matching, directory scoping, 100-file limit

Shell Execution

Tool	Description	Key Features
Bash (`BashInput`)	Shell command execution	`timeout` (max 600s), `run_in_background`, `dangerouslyDisableSandbox`, structured content output
TaskOutput	Read background task output	`block`, `timeout`, task_id
TaskStop	Stop background tasks	task_id or shell_id

Web & External

Tool	Description	Key Features
WebSearch	Search the web	`allowed_domains`, `blocked_domains`
WebFetch	Fetch and process URLs	`url`, `prompt` for content processing

MCP Integration

Tool	Description	Key Features
Mcp (`McpInput`)	Execute MCP tools	Dynamic schema from servers
ListMcpResources	List available resources	Server filtering
ReadMcpResource	Read MCP resources	URI-based, blob support

Agent & Workflow

Tool	Description	Key Features
Agent (`AgentInput`)	Spawn subagents	`run_in_background`, `isolation: "worktree"`, permission `mode`, named agents
TodoWrite	Task list management	`pending`/`in_progress`/ `completed` states
AskUserQuestion	Interactive user questions	1-4 questions, multi-select, preview content, annotations
ExitPlanMode	Plan mode exit	`allowedPrompts` for plan-based permissions

Git Worktree

Tool	Description	Key Features
EnterWorktree	Create/switch git worktrees	Named worktrees, path-based switching

Tool	Description	Key Features
ExitWorktree	Leave worktrees	`keep` or `remove` with change protection

Tool Result Persistence Layer

A critical innovation: **tool results too large for inline context are persisted to disk**: - `persistedOutputPath` / `persistedOutputSize` - large outputs saved to `tool-results` dir - `rawOutputPath` - MCP tool raw outputs - MCP tool results can bypass token-based persist layer via `_meta["anthropic/maxResultSizeChars"]` (up to 500K)

Progress System

```
// Progress message format (from binary)
{type: "progress", data: {...}, toolUseID, parentToolUseID,
         uuid, timestamp}
```

Progress types include: - Hook progress (`hook_progress`) - Tool execution progress - Streaming deltas - Attachment progress

---

4. Context Management System

Auto-Compaction (The “Infinite Conversation” Feature)

Claude Code implements **automatic conversation compaction** to handle arbitrarily long sessions:

How it works: 1. Monitors token usage against `CLAUDE_CODE_MAX_CONTEXT_TOKENS` 2. When approaching limits (80% threshold), automatically compacts history 3. Preserves essential context while summarizing older turns 4. Re-executes skills against the next user message if invoked before compaction

Technical details from CHANGELOG: - `DISABLE_COMPACT` env var to disable auto-compaction - Auto-compact threshold increased from 60% to 80% - Compaction detects thrashing loops (3 consecutive compactions with no progress = error) - Empty hook entries skipped to reduce transcript size - Pre-edit file copies capped to prevent bloat

Context Management Metadata

From the binary strings, each assistant message carries:

```
context_management: {
  // Token usage tracking
  // Cache hit/miss information
  // Compaction markers
}
```

Token Usage & Caching

Claude Code has sophisticated caching: - **Prompt caching** with Anthropic API (ephemeral 1h and 5m cache tiers) - Cache creation/read token tracking in tool outputs - Pro users see footer hints when prompt cache expires - `/context` command shows free space and message breakdowns

Settings Hierarchy for Context

```
managed-settings.json (enterprise policy)
  → ~/.claude/settings.json (user)
    → .claude/settings.json (project)
      → CLI flags (highest priority)
```

---

5. File Editing Capabilities

Edit Tool (`FileEditInput`)

Format: `old_string` / `new_string` / `replace_all`

Innovations: - Shorter `old_string` anchors to reduce output tokens (optimization in v2.1.91) - Works on files viewed via `Bash` with `sed -n` or `cat` without requiring separate `Read` - Tracks user modifications in permission dialog (`userModified: boolean`) - Structured patch output with `oldStart`, `oldLines`, `newStart`, `newLines`

Write Tool (`FileWriteInput`)

Format: `file_path` (absolute) + `content`

Features: - Git diff output with additions/deletions/changes - `type: "create" | "update"` - Original file preservation for diff generation - User modification tracking

Diff Display System

- **Word-level diff** display for improved readability
- **GitHub-style patch** output in tool results
- Write tool diff computation optimized: 60% faster on files with tabs/&/$
- Diffs disappear from UI on `--resume` for files >10KB (fixed in recent versions)

Protected Paths

Default protected directories (require explicit approval): - `.claude/`, `.git/`, `.vscode/`, `.husky/` - Shell config files - `.claude/skills/`, `.claude/agents/`, `.claude/commands/` (with `--dangerously-skip-permissions`)

---

6. Shell Command Execution (Bash Tool)

Architecture

The Bash tool is the most security-critical component:

Input Schema:

```
{
  command: string;           // The command to execute
  description?: string;      // Human-readable description (5-10
        words for simple, longer for complex)
  timeout?: number;           // Max 600000ms (10 minutes)
  run_in_background?: boolean;
  dangerouslyDisableSandbox?: boolean;
}
```

Output Schema:

```
{
  stdout: string;
  stderr: string;
  interrupted: boolean;
  backgroundTaskId?: string;
  backgroundedByUser?: boolean;      // Ctrl+B backgrounding
  assistantAutoBackgrounded?: boolean;
  dangerouslyDisableSandbox?: boolean;
  returnCodeInterpretation?:
        string;  // Semantic meaning of exit codes
  noOutputExpected?: boolean;
  structuredContent?: unknown[];      // Rich output blocks
  persistedOutputPath?: string;        // Large output disk path
  persistedOutputSize?: number;
  staleReadFileStateHint?: string;    // Warning when files
        changed during command
  ghRateLimitHint?: string;           // GitHub API rate limit
        helper
}
```

Permission Modes (5 Modes)

Mode	Description
`default`	Standard approval workflow
`auto`	Auto-approves safe commands, prompts for dangerous ones
`acceptEdits`	Auto-approves file edits, asks for everything else
`dontAsk`	Auto-approves read-only commands
`bypassPermissions`	Auto-approves everything (dangerous)

Mode	Description
`plan`	Plan mode - requires plan approval before execution

Permission Rules System

Sophisticated rule-based permissions in `settings.json`:

```
{
  "permissions": {
    "ask": ["Bash"],
    "deny": ["WebSearch", "WebFetch"],
    "defaultMode": "auto",
    "allow": ["Bash(git status:*)", "Bash(git add:*)",
        "Bash(ls:*)", "Read(*)", "Edit(*)", "Write(*)"],
    "deny": ["Bash(rm -rf *)", "Bash(curl -X POST *)"]
  }
}
```

Rule Features: - Wildcard patterns: `Bash(git commit:*)` - Tool-specific: `Read`, `Edit`, `Write`, `Bash` - Compound command handling (`ls && git push`) - Env-var prefix handling (`FOO=bar git push`) - Read-only command auto-detection

Sandboxing

```
{
  "sandbox": {
    "enabled": true,
    "autoAllowBashIfSandboxed": false,
    "allowUnsandboxedCommands": false,
    "excludedCommands": [],
    "network": {
      "allowUnixSockets": [],
      "allowAllUnixSockets": false,
      "allowLocalBinding": false,
      "allowedDomains": [],
      "httpProxyPort": null,
      "socksProxyPort": null
    },
    "enableWeakerNestedSandbox": false
  }
}
```

Linux Sandboxing: - PID namespace isolation when `CLAUDE_CODE_SUBPROCESS_ENV_SCRUB` is set - `apply-seccomp` helper for syscall filtering - Unix socket blocking - Network domain restrictions - `CLAUDE_CODE_SCRIPT_CAPS` for per-session script invocation limits

Bash Tool Safety Checks: - Dangerous `rm` operation detection - Network redirect detection (`/dev/tcp/...`, `/dev/udp/...`) - Backslash-escaped flag detection - Piped command analysis - Archive extraction TOCTOU protection - PowerShell argument-splitting hardening

Background Task Management

- `run_in_background` flag for long-running commands
- `Ctrl+B` to background running commands interactively
- `TaskOutput` tool to read background task results
- `TaskStop` tool to cancel background tasks
- Progress messages based on last 5 lines of output
- Assistant can auto-background long blocking commands

---

7. Git Integration

Worktree-Based Isolation

Claude Code has **first-class git worktree support**:

EnterWorktree Tool: - Create named worktrees with validation (`/[a-zA-Z0-9._-]+/`, max 64 chars) - Switch to existing worktrees from `git worktree list` - Isolated working copies for subagents

ExitWorktree Tool: - `keep`: Leaves worktree and branch on disk - `remove`: Deletes both, with change protection (requires `discard_changes: true` for uncommitted files)

Git Workflow Features

- `claude project purge [path]` - Delete all Claude Code state for a project
- `--from-pr` accepts GitHub, GitLab, Bitbucket PR URLs
- Pasting PR URL into `/resume` finds the session that created that PR
- Git status in startup logo (unless `CLAUDE_CODE_HIDE_CWD`)
- `workspace.git_worktree` in status line JSON
- Subagents with `isolation: "worktree"` work on isolated copies

GitHub Integration

- `gh pr create` support via commit-commands plugin
- GitHub API rate limit detection with contextual hints
- `owner/repo#N` shorthand links use git remote's host
- `prUrlTemplate` for custom code-review URLs
- `AI_AGENT` env var set for subprocesses so `gh` can attribute traffic

---

8. MCP (Model Context Protocol) Support

MCP Server Types Supported

Transport	Description
stdio	Local process - stdin/stdout communication
SSE	Server-Sent Events - cloud services
HTTP	Streamable HTTP - newer spec

Transport	Description
WebSocket	WebSocket transport

MCP Configuration

`.mcp.json` **format (project-level):**

```
{
  "server-name": {
    "command": "npx",
    "args": ["-y", "@modelcontextprotocol/server-filesystem", "/
        allowed/path"],
    "env": {"LOG_LEVEL": "debug"}
  }
}
```

Plugin-level MCP (in `plugin.json`**):**

```
{
  "mcpServers": {
    "plugin-api": {
      "command": "${CLAUDE_PLUGIN_ROOT}/servers/api-server",
      "args": ["--port", "8080"]
    }
  }
}
```

MCP Features

- **OAuth 2.0** support with Authorization Server discovery
- **Auto-reconnection** for SSE connections on disconnect
- `alwaysLoad` option - tools skip deferred loading and are always available
- **Tool annotations** and titles display in `/mcp` view
- **Resource mentions** via @ typeahead
- **Non-blocking connections** (`MCP_CONNECTION_NONBLOCKING=true`) for `-p` mode
- **claude.ai connectors** - built-in MCP servers from Claude.ai
- **Headers helper** scripts for dynamic auth
- Server startup timeout: `MCP_TIMEOUT` env var
- Connection bounded at 5s for `--mcp-config`

Claude Code as MCP Server

`claude mcp serve` - Claude Code can act as an MCP server itself, exposing its tools to other MCP clients.

---

9. Plugin System Architecture

Plugin Structure

```
plugin-name/
├── .claude-plugin/
│   └── plugin.json          # Plugin metadata
├── commands/                # Slash commands (.md files with YAML frontma
├── agents/                  # Specialized agents (.md files)
├── skills/                  # Skills (SKILL.md files)
├── hooks/                   # Event handlers (hooks.json + scripts)
├── themes/                  # Custom themes (JSON files)
├── .mcp.json                # MCP server configs
└── bin/                     # Plugin executables
```

Plugin Metadata (`plugin.json`)

```
{
  "name": "hookify",
  "version": "0.1.0",
  "description": "Easily create hooks to prevent unwanted
        behaviors",
  "author": {"name": "...", "email": "..."},
  "mcpServers": {...}        // Optional inline MCP
}
```

Slash Commands

Commands are **Markdown files with YAML frontmatter** in `.claude/commands/`:

```
---
allowed-tools: Bash(git:*), Read(*)
description: Commit, push, and open a PR
---

## Context
- Current git status: !`git status`
- Current branch: !`git branch --show-current`

## Your task
1. Create a new branch if on main
2. Create a single commit...
```

Features: - Inline shell execution with !`command` syntax - @-mention file autocomplete - Tool restriction via `allowed-tools` frontmatter - Namespacing: `.claude/commands/frontend/component.md` → `/frontend:component` - `keep-coding-instructions` frontmatter for output styles

Skills

Skills are **Markdown files with YAML frontmatter** in `skills/<name>/SKILL.md`:

```
---
name: frontend-design
description: Create distinctive, production-grade frontend
        interfaces
license: Complete terms in LICENSE.txt
---

# Skill content...
```

Skill invocation triggers: - User-typed slash commands - Claude proactive invocation - Nested skill invocation

Skill features: - `${CLAUDE_EFFORT}` variable substitution - `context: fork` for isolated skill execution - Agent frontmatter for delegating to specialized agents - Auto-invoked based on conversation context

Agents

Agents are specialized LLM configurations in `agents/<name>.md`:

```
---
name: code-explorer
description: Deeply analyzes existing codebase features
 tools: Glob, Grep, LS, Read, NotebookRead, WebFetch, TodoWrite,
        WebSearch, KillShell, BashOutput
model: sonnet
color: yellow
permissionMode: plan
---

# Agent system prompt...
```

Agent features: - Tool restriction per-agent - Model override (`sonnet`/`opus`/`haiku`) - Color theming - Permission mode inheritance - Worktree isolation - Named subagents addressable via `SendMessage({to: name})`

Hooks System

Hooks are **event-driven scripts** that intercept tool execution and conversation flow:

Hook Events: | Event | When Fired | Blocking? | |——-|————|————| | **PreToolUse** | Before any tool executes | Yes - can deny/block | | **PostToolUse** | After tool succeeds | No - can add context | | **PostToolUseFailure** | After tool fails | No | | **Stop** | When user tries to exit | Yes - can prevent exit | | **SubagentStop** | When subagent tries to exit | Yes | | **UserPromptSubmit** | When user sends a message | Yes | | **PreCompact** | Before context compaction | No | | **TaskCreated** | When background task created | Yes | | **PermissionDenied** | After auto-mode denial | No - can request retry |

Hook Configuration (`hooks.json`):

```
{
  "hooks": {
    "PreToolUse": [
      {
        "hooks": [
          {
            "type": "command",
            "command": "python3 ${CLAUDE_PLUGIN_ROOT}/hooks/
        pretooluse.py",
            "timeout": 10
          }
        ]
      }
    ]
  }
}
```

Hook Input Format:

```
{
  "tool_name": "Bash",
  "tool_input": {"command": "rm -rf /tmp/test"},
  "hook_event_name": "PreToolUse",
  "transcript_path": "/path/to/transcript.jsonl"
}
```

Hook Output Format:

```
{
  "decision": "block",
  "reason": "Dangerous rm command!",
  "systemMessage": "...",
  "hookSpecificOutput": {
    "hookEventName": "PreToolUse",
    "permissionDecision": "deny",
    "updatedToolOutput": "..."
  }
}
```

Advanced Hook Features: - `type: "mcp_tool"` - Hooks can invoke MCP tools directly - `duration_ms` in PostToolUse/PostToolUseFailure inputs - Hook output >50K saved to disk with file path + preview - Async hooks supported - `if` condition filtering for commands with env-var prefixes - `"defer"` permission decision for headless sessions

Themes

Custom themes stored in `~/.claude/themes/` or shipped via plugin `themes/` directory: - Named themes selectable via `/theme` - JSON format - Plugins can ship themes

10. TUI/UX Implementation

Rendering Modes

Claude Code supports **two rendering modes:**

1. Fullscreen Mode (Default)

- Uses terminal alt-screen (DEC 2026 support detection)
- Virtualized scrollback
- Mouse support (clicking, scrolling, URL opening)
- Dialogs, pickers, overlays
- Vim mode (v/V visual mode, j/k navigation)

2. No-Flicker Mode (`NO_FLICKER=1` or `CLAUDE_CODE_NO_FLICKER=1`)

- Flicker-free alt-screen rendering
- Virtualized scrollback
- Focus view toggle (`Ctrl+O`) - shows prompt + one-line tool summary + final response
- Designed for terminals with poor alt-screen handling

Interactive Components

Slash Command Picker: - Type-to-filter search - Contiguous substring highlighting - Long descriptions wrap to second line - Namespace support (`/frontend:component`)

Model Picker (`/model`): - Lists models from gateway's `/v1/models` endpoint when using custom base URL - Effort level selection - Model description overrides via env vars

Skills Picker (`/skills`): - Type-to-filter search box - Plugin-provided skills auto-loaded

Agents View (`/agents`): - Tabbed layout: Running tab + Library tab - Run agent and View running instance actions - `● N running` indicator per agent type

Resume Picker (`/resume`): - Cross-project session resume - Filter with project/worktree/branch names - PR URL paste finds creating session - Parallel loading for all-projects view

Progress & Status System

Spinner System: - Token count indication in spinner - Tool usage count display - Time-based spinner tips (customizable via `spinnerTipsOverride`) - Red spinner when permission check stalls

Status Line (`/status-line`): - Custom status line command support - `refreshInterval` for periodic re-run - JSON input includes `workspace.git_worktree`, `effort.level`, `thinking.enabled`

Notifications: - Transient footer notifications (e.g., context-low warning) - `invalidates` property for clearing notifications - Task completion notifications for background agents

Input System

Keybindings: - Custom keybindings in `~/.claude/keybindings.json` - Vim mode (NORMAL/INSERT, c, f/F, t/T, visual mode) - `Ctrl+n/p` or `j/k` for menu navigation - `Shift+Enter` for newline insertion - `Ctrl+L` forces screen redraw - `Ctrl+Z` suspends process

Paste Handling: - Bracketed paste support - Kitty keyboard protocol sequences - CRLF content normalization - Image paste with automatic downscaling to 2000px - Multi-line paste support

Streaming Architecture

Streaming Display: - Real-time assistant message streaming - Tool use streaming with progress indicators - Image rendering during streaming - CJK/Unicode text handling - Combining-mark text support (Devanagari)

Stream Resilience: - Stalled streaming falls back to non-streaming mode - "Stream idle timeout" recovery (fixed for sleep/wake, background sessions) - Retry with exponential backoff for 429s - API retry countdown display

---

11. Configuration System

Settings Precedence Hierarchy

```
1. CLI flags (highest)
2. Environment variables
3. .claude/settings.json (project-level)
4. ~/.claude/settings.json (user-level)
5. managed-settings.json (enterprise policy, lowest)
```

Key Settings

```
{
  "permissions": {
    "defaultMode": "auto",
    "ask": ["Bash"],
    "deny": ["WebSearch"],
    "allow": ["Bash(git status:*)"],
    "disableBypassPermissionsMode": "disable"
  },
  "sandbox": {
    "enabled": true,
    "network": {
      "allowedDomains": ["api.example.com"],
      "allowLocalBinding": false
    }
  },
  "mcpServers": {...},
  "hooks": {...},
```

```
  "cleanupPeriodDays": 30,
  "showThinkingSummaries": false,
  "language": "en",
  "theme": "default",
  "editorMode": "vim",
  "verbose": false,
  "prUrlTemplate": "https://github.com/{owner}/{repo}/pull/
       {number}"
}
```

Managed Settings (Enterprise)

- `forceRemoteSettingsRefresh` - Blocks startup until remote settings fetched
- `allowManagedPermissionRulesOnly` - Only managed rules apply
- `allowManagedHooksOnly` - Only managed hooks allowed
- `blockedMarketplaces` - Enforce marketplace restrictions
- `wslInheritsWindowsSettings` - WSL inherits Windows-side managed settings

---

12. Unique Innovations & Power Features

1. Bun-Compiled Native Binary Distribution

Claude Code is a ~500MB JS app compiled to native binaries using Bun. This gives: - Single-file distribution (no Node.js dependency for users) - Fast startup (no npm install overhead) - Cross-platform native performance - Embedded V8 engine with full Node.js compatibility

2. Auto-Compaction for Infinite Conversations

Automatically manages context window by compacting old conversation turns. Detects thrashing loops and stops with actionable errors instead of burning API calls.

3. Permission Mode System with Granular Rules

Five permission modes plus wildcard rule syntax (`Bash(git commit:*)`), compound command handling, and env-var prefix support. The most sophisticated permission system in any CLI agent.

4. Git Worktree-Based Agent Isolation

Subagents can work in isolated git worktrees with automatic cleanup. Enables parallel development workflows without branch conflicts.

5. Hook-Based Extensibility Architecture

Event-driven plugin system with 9+ hook events. Hooks can block tool execution, modify outputs, prevent session exit, and even invoke MCP tools. Python/JS/shell script support.

6. Multi-Model/Multi-Provider Backend

Supports Anthropic API, Bedrock (with Mantle), Vertex AI, Azure, and custom gateways. Interactive setup wizards for complex auth flows.

7. Tool Result Persistence Layer

Large tool outputs (>50K chars) automatically saved to disk with file paths instead of cluttering context. MCP tools can bypass with `_meta` annotations (up to 500K).

8. Background Task System with Ctrl+B

Commands can run in background with progress tracking. Interactive Ctrl+B backgrounding. Parent agents can check child progress via `TaskOutput`.

9. No-Flicker Alt-Screen Rendering Mode

Dedicated rendering mode for terminals with poor alt-screen support. Virtualized scrollback, focus view toggle, reduced redraw artifacts.

10. Plan Mode with Structured Approval

Plan mode generates structured plans with `ExitPlanMode` tool. Plans saved to files, can be edited by user. `allowedPrompts` for semantic action categories.

11. MCP Deep Integration

Full MCP lifecycle: stdio/SSE/HTTP/WebSocket transports, OAuth 2.0, auto-reconnection, non-blocking startup, resource mentions, claude.ai connectors. Can also serve as MCP server.

12. Context-Aware Skill System

Skills auto-invoke based on conversation context. Support `context: fork` for isolated execution, `${CLAUDE_EFFORT}` variable substitution, agent delegation, and user/proactive/nested trigger attribution.

13. Session Transcript System

Full conversation persistence in JSONL format. Supports `--resume`, `--continue`, cross-project resume, PR URL-based session finding. Corrupt lines skipped gracefully.

14. LSP (Language Server Protocol) Integration

Claude Code can connect to LSP servers for diagnostics. Identifies itself via `clientInfo` in initialize requests. Shows diagnostic summaries expandable on click.

15. OpenTelemetry Integration

Full OTEL tracing with `claude_code.*` event names. W3C TRACEPARENT propagation to subprocesses. Resource attributes for OS, arch, WSL version.

16. Voice Mode

Push-to-talk voice dictation with microphone permission handling. Respects VS Code `accessibility.voice.speechLanguage` setting.

17. Remote Control

Connect local terminal to claude.ai/code web sessions. Session names with hostname prefix. Real-time synchronization.

18. PowerShell Tool on Windows

Full PowerShell support on Windows (5.1 and 7+). Version-appropriate syntax guidance. Argument-splitting hardening. Primary shell when PowerShell tool enabled.

19. Subprocess Sandboxing (Linux)

PID namespace isolation, seccomp syscall filtering, network restrictions, unix socket blocking, per-session script capability limits.

20. Smart File Editing

Edit tool works on files viewed via Bash without separate Read. Tracks user modifications. Word-level diff display. Git diff output. 60% faster diff computation via algorithmic optimization.

---

13. Performance Optimizations

From CHANGELOG Analysis

1. **Bun.stripANSI routing** - Replaced custom stripAnsi with Bun's native implementation
2. **Eliminated per-turn JSON.stringify** of MCP tool schemas on cache-key lookup
3. **SSE transport** - Large streamed frames handled in linear time (was quadratic)
4. **SDK transcript writes** - No longer slow down quadratically on long conversations
5. **Image handling** - Compressed to same token budget as Read tool, downscaled to 2000px
6. **Memory leak fixes** - Multiple fixes for unbounded growth (images, `/usage`, MCP buffers, hook handlers)
7. **File descriptor management** - Bounded usage during `find` in Bash tool on large trees
8. **Parallel loading** - Resume picker loads project sessions in parallel
9. **Edit tool optimization** - Shorter old_string anchors, 60% faster diff computation
10. **StructuredOutput schema cache** - Fixed ~50% failure rate with multiple schemas

---

14. Error Recovery & Edge Case Handling

Streaming Resilience

- Stalled streaming → non-streaming fallback
- Sleep/wake stream idle timeout recovery
- Background session false aborts during thinking pauses fixed
- API retry with exponential backoff (minimum enforced)

Session Recovery

- Corrupt transcript lines skipped on `--resume`
- `--resume` cache miss recovery
- Attachment messages persisted even when dialogs appear
- Messages typed during processing enqueued properly

Tool Resilience

- Grep tool falls back to system `rg` when embedded binary stale
- Bash tool warns when formatter modifies previously-read files
- Edit tool handles format-on-save hooks between consecutive edits
- MCP servers auto-retry transient errors up to 3 times

Permission Edge Cases

- `toString` prototype property names in settings no longer break parsing
- JavaScript prototype property names in permission rules handled
- Wildcard rules match commands with extra spaces/tabs
- Piped commands with `cd` segments handled correctly
- Redirects to `/dev/tcp/...` properly blocked

---

15. SDK & API Integration

TypeScript SDK

```
import { claude } from '@anthropic-ai/claude-code';
```

Python SDK

```
# pip install claude-code-sdk
```

SDK Features

- `mcp_authenticate` with `redirectUri` support
- `read_file` with size cap enforcement
- `--fork` subagent support (`CLAUDE_CODE_FORK_SUBAGENT=1`)
- Bridge sessions showing local git repo info on claude.ai session cards
- `--print` mode with `--output-format=stream-json`
- `--exclude-dynamic-system-prompt-sections` for cross-user prompt caching

---

16. Top 20 Most Important Features to Port to HelixCode

Priority 1: Core Architecture (Must-Have)

1. **Auto-Compaction System**
   - What: Automatic context window management with summarization
   - Why: Enables arbitrarily long sessions without manual `/compact`
   - Port: Implement token counting + summarization pipeline with thrashing detection

    - Complexity: HIGH
2. **Permission Rule System with 5 Modes**
    - What: Granular tool approval with wildcard rules, compound command handling
    - Why: Critical for security and UX - balances autonomy with safety
    - Port: Rule engine with regex matching, env-var prefix handling, tool-specific patterns
    - Complexity: HIGH
3. **Tool Result Persistence Layer**
    - What: Auto-save large outputs to disk, reference by path in context
    - Why: Prevents context pollution, enables large file/directory listings
    - Port: File-based result storage with path references in message format
    - Complexity: MEDIUM
4. **Git Worktree-Based Agent Isolation**
    - What: Spawn subagents in isolated git worktrees with auto-cleanup
    - Why: Enables parallel development without branch conflicts
    - Port: Git worktree management + subagent process isolation
    - Complexity: HIGH
5. **Hook-Based Extensibility**
    - What: Event-driven plugin hooks (PreToolUse, PostToolUse, Stop, etc.)
    - Why: Most powerful customization mechanism - can block, modify, extend behavior
    - Port: Event bus + external process invocation with JSON I/O
    - Complexity: HIGH

Priority 2: UX & Productivity (High-Impact)

1. **No-Flicker Rendering Mode**
    - What: Alternative TUI renderer for terminals with poor alt-screen support
    - Why: Critical compatibility feature for broad terminal support
    - Port: Separate renderer implementation with virtualized scrollback
    - Complexity: HIGH
2. **Background Task System (Ctrl+B)**
    - What: Run commands in background with progress tracking and retrieval
    - Why: Essential for long-running builds, tests, deployments
    - Port: Process management + task queue + progress streaming
    - Complexity: MEDIUM
3. **Smart File Editing (Edit without Read)**
    - What: Edit files viewed via Bash without separate Read tool call
    - Why: Reduces token usage and tool call count
    - Port: Track file content from Bash outputs, make available to Edit tool
    - Complexity: MEDIUM
4. **Plan Mode with Structured Approval**
    - What: Generate plans with semantic action categories, require approval
    - Why: Enables high-trust workflows while maintaining oversight
    - Port: Plan generation + approval workflow + allowedPrompts tracking
    - Complexity: MEDIUM
5. **Slash Command System with Frontmatter**
    - What: Markdown files with YAML frontmatter become `/commands`
    - Why: Easy customization, version-controllable prompts, inline shell execution
    - Port: Markdown parser + frontmatter extraction + command registry
    - Complexity: LOW

Priority 3: Integration & Ecosystem (Differentiating)

1. **MCP Full Lifecycle Support**
   - What: stdio/SSE/HTTP/WebSocket transports, OAuth, auto-reconnection
   - Why: Ecosystem integration - connects to databases, APIs, file systems
   - Port: MCP client implementation per transport + OAuth flow + server management
   - Complexity: VERY HIGH
2. **Skill System with Auto-Invocation**
   - What: Context-aware skill loading with variable substitution and fork isolation
   - Why: Domain-specific expertise without manual activation
   - Port: Skill registry + trigger detection + isolated execution context
   - Complexity: HIGH
3. **Session Transcript with Cross-Project Resume**
   - What: JSONL persistence, `--resume`, `--continue`, PR URL-based finding
   - Why: Conversation continuity is critical for complex tasks
   - Port: Transcript serialization + session index + search/resume logic
   - Complexity: MEDIUM
4. **Multi-Provider Backend**
   - What: Anthropic, Bedrock, Vertex, Azure, custom gateways with interactive setup
   - Why: Deployment flexibility, enterprise requirements
   - Port: Provider abstraction layer + auth adapters + setup wizards
   - Complexity: HIGH
5. **LSP Diagnostic Integration**
   - What: Connect to language servers, show expandable diagnostics
   - Why: Real-time code quality feedback
   - Port: LSP client + diagnostic aggregation + display integration
   - Complexity: MEDIUM
6. **Sandboxed Shell Execution**
   - What: PID namespace isolation, seccomp, network restrictions
   - Why: Security for arbitrary code execution
   - Port: Linux namespaces + seccomp BPF + network filtering
   - Complexity: VERY HIGH
7. **Theme System with Custom Themes**
   - What: Named themes in JSON, plugin-shipped themes
   - Why: Accessibility, personalization, branding
   - Port: Theme registry + color system + terminal color mapping
   - Complexity: LOW
8. **AskUserQuestion with Previews**
   - What: Interactive questions with option previews, multi-select, annotations
   - Why: Rich user interaction for decisions
   - Port: Dialog component + option rendering + preview content
   - Complexity: MEDIUM
9. **Subagent Team with Named Agents**
   - What: Named, addressable subagents with SendMessage, team context
   - Why: Multi-agent collaboration workflows
   - Port: Agent registry + message routing + team context sharing
   - Complexity: HIGH
10. **OpenTelemetry Integration**
    - What: Full tracing with W3C propagation, subprocess span inheritance
    - Why: Observability for production deployments
    - Port: OTEL SDK integration + span management + context propagation
    - Complexity: MEDIUM

Appendix A: Complete Tool Schemas Summary

Input Schemas

- `AgentInput`: description, prompt, subagent_type, model, run_in_background, name, team_name, mode, isolation
- `BashInput`: command, description, timeout, run_in_background, dangerouslyDisableSandbox
- `FileEditInput`: file_path, old_string, new_string, replace_all
- `FileReadInput`: file_path, offset, limit, pages
- `FileWriteInput`: file_path, content
- `GlobInput`: pattern, path
- `GrepInput`: pattern, path, glob, output_mode, -A, -B, -C, context, -n, -i, type, head_limit, offset, multiline
- `ListMcpResourcesInput`: server
- `McpInput`: [dynamic schema per server]
- `NotebookEditInput`: notebook_path, cell_id, new_source, cell_type, edit_mode
- `ReadMcpResourceInput`: server, uri
- `TodoWriteInput`: todos[]
- `WebFetchInput`: url, prompt
- `WebSearchInput`: query, allowed_domains, blocked_domains
- `AskUserQuestionInput`: questions[]
- `EnterWorktreeInput`: name, path
- `ExitWorktreeInput`: action, discard_changes

Output Schemas

- `AgentOutput`: content, usage, toolStats, status, prompt (or async_launched)
- `BashOutput`: stdout, stderr, interrupted, backgroundTaskId, structuredContent, persistedOutputPath
- `FileEditOutput`: filePath, oldString, newString, structuredPatch, gitDiff
- `FileReadOutput`: text/image/notebook/pdf/parts/file_unchanged variants
- `FileWriteOutput`: type, filePath, content, structuredPatch, gitDiff
- `GlobOutput`: durationMs, numFiles, filenames, truncated
- `GrepOutput`: mode, numFiles, filenames, content, numLines, numMatches
- `McpOutput`: string
- `NotebookEditOutput`: new_source, cell_id, cell_type, language, edit_mode
- `WebFetchOutput`: bytes, code, codeText, result, durationMs
- `WebSearchOutput`: [search results]

Appendix B: Environment Variables

API Configuration

- `ANTHROPIC_API_KEY` / `ANTHROPIC_AUTH_TOKEN`
- `ANTHROPIC_BASE_URL` / `ANTHROPIC_BEDROCK_BASE_URL`
- `ANTHROPIC_MODEL` / `ANTHROPIC_SMALL_FAST_MODEL`
- `ANTHROPIC_DEFAULT_*_MODEL_NAME` / `_DESCRIPTION`
- `ANTHROPIC_BEDROCK_SERVICE_TIER`
- `AWS_BEARER_TOKEN_BEDROCK`
- `AWS_REGION` / `AWS_DEFAULT_REGION`

Runtime Behavior

- CLAUDE_CODE_MAX_CONTEXT_TOKENS
- DISABLE_COMPACT
- CLAUDE_CODE_DISABLE_EXPERIMENTAL_BETAS
- CLAUDE_CODE_DISABLE_NONESSENTIAL_TRAFFIC
- DISABLE_TELEMETRY
- DISABLE_AUTOUPDATER / DISABLE_UPDATES
- CLAUDE_CODE_FORK_SUBAGENT
- CLAUDE_CODE_HIDE_CWD
- CLAUDE_CODE_NO_FLICKER
- CLAUDE_CODE_GIT_BASH_PATH
- CLAUDE_CODE_ORGANIZATION_UUID
- CLAUDE_CODE_PERFORCE_MODE
- CLAUDE_CODE_SUBPROCESS_ENV_SCRUB
- CLAUDE_CODE_SCRIPT_CAPS

Sandbox & Permissions

- --dangerously-skip-permissions
- CLAUDE_CODE_USE_MANTLE

MCP

- MCP_TIMEOUT / MCP_TOOL_TIMEOUT
- MCP_CONNECTION_NONBLOCKING

Timing

- API_TIMEOUT_MS
- BASH_DEFAULT_TIMEOUT_MS / BASH_MAX_TIMEOUT_MS

Appendix C: File Structure (Runtime)

```
~/.claude/
├── settings.json              # User settings
├── keybindings.json           # Custom keybindings
├── themes/                    # Custom themes
├── history.jsonl              # Prompt history
├── CLAUDE.md                  # Global context
└── [session storage]          # Transcripts, tasks, file history

.claude/ (project-level)
├── settings.json              # Project settings
├── commands/                  # Custom slash commands
├── skills/                    # Project skills
├── agents/                    # Project agents
├── hooks/                     # Project hooks
└── CLAUDE.md                # Project context
```

Analysis completed from binary string analysis, SDK type definitions, CHANGELOG review (3412 lines, 120+ versions), plugin architecture examination, and installation script reverse engineering.